

Day 54: Batch Normalization in CNNs

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 8: Classic CNN Architectures

0. Where Today Fits

Day 40 introduced batch normalization on dense layers: normalize each feature to zero mean/unit variance across the batch, then let a learned gamma/beta rescale and re-shift it. Today extends that to conv layers. Day 53 left a real cliffhanger: both PlainDeepConvNet and ResNetStyleConvNet (7 conv layers) failed completely at $\text{lr}=0.15$. Does batch norm fix that, alone or combined with residual connections?

Start here (no prior knowledge assumed):

A conv layer's output has shape (batch, channels, height, width). Dense-layer batch norm (Day 40) treats each of D features as independent, normalizing each across the batch dimension. For a conv layer, "one feature" is now one CHANNEL -- so today's spatial batch norm normalizes each channel using statistics pooled over the batch AND both spatial axes (every pixel of every image in the batch, for that channel), with one learned gamma/beta per channel, not per pixel.

Watch: [Batch Normalization \("batch norm"\) explained](#)

Watch: [Batch Normalization | Internal Covariate Shift | Deep Learning Part 8](#)

1. Spatial Batch Norm, Exactly

MATH

Given a conv layer's output Z , shape (m, C, H, W) :

$\mu = Z.\text{mean}(\text{axis}=(0,2,3))$ -- per-channel mean, pooled over batch+H+W

$\text{var} = Z.\text{var}(\text{axis}=(0,2,3))$ -- per-channel variance, same pooling

$\hat{Z} = (Z - \mu) / \sqrt{\text{var} + \text{eps}}$ -- normalize to zero mean, unit variance

$\text{out} = \gamma * \hat{Z} + \beta$ -- learned per-channel rescale + shift, gamma/beta shape $(1, C, 1, 1)$

Day 40's dense version pooled over axis=0 only ($N=m$ values per feature); today pools over axis=(0,2,3) ($N=m*H*W$ values per channel) -- the only real change.

SYSTEM DIAGRAM

Placement matters: conv -> BN -> ReLU, not conv -> ReLU -> BN. BN normalizes the RAW pre-activation (before the non-linearity clips half of it to zero), matching real BatchNorm2d placement in every standard CNN architecture, including ResNet.

2. A Real Bug Caught by the Gradient Check

The first version of `spatial_batchnorm_backward` computed $d\gamma$ and $d\beta$ as a raw sum over (batch, H, W) with no further scaling. The gradient check failed with every gamma/beta entry's analytic value coming out exactly 4x (later, in the real 7-layer networks, exactly m -times) too large. The cause: every OTHER parameter gradient in this codebase's conv/FC layers (dW , db) is explicitly divided by the batch size m to match `cross_entropy_loss`'s own "mean over the batch" convention -- $d\gamma/d\beta$ needed that same $/m$ division, which the original derivation

omitted.

ACTUAL OUTPUT

```
max relative error across all checked entries: 5.60e-05 (PASS -- spatial batch norm + residual
backward chain is correct)
```

3. Gradient Flow by Depth -- Three Networks, Same Initial Weights

Day 53 measured gradient norm by depth for plain vs. residual networks. Today repeats that exact diagnostic with a third network: BN with no shortcut. All three start from bit-identical initial conv/FC weights.

ACTUAL OUTPUT

```
stem-to-deepest-layer gradient ratio --
plain: 0.1159   BN-only: 1.5335   BN+residual: 1.1514
```

BN changes gradient distribution far more dramatically than residual shortcuts did on their own. BN-only's ratio actually EXCEEDS 1.0 -- the stem's gradient is larger than the deepest block's, meaning batch norm doesn't just preserve gradient magnitude across depth, it can actively rebalance which layers receive more of it.

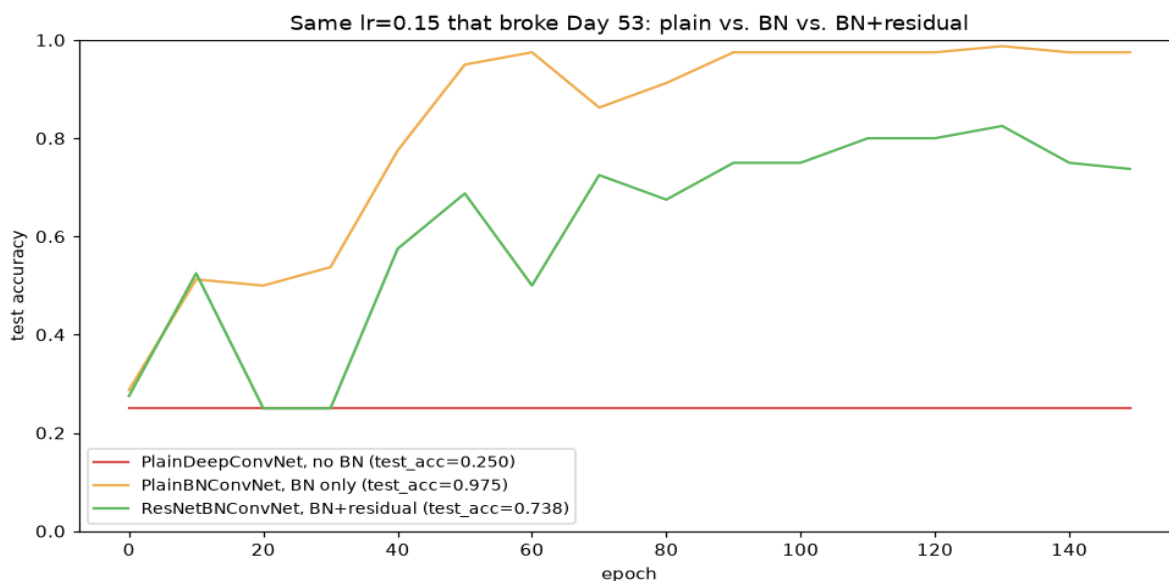
4. The Trained Comparison -- Same $\text{lr}=0.15$ That Broke Day 53

ACTUAL OUTPUT

```
network params train_acc test_acc
PlainDeepConvNet 7012 0.2500 0.2500
PlainBNConvNet 7124 1.0000 0.9750
ResNetBNConvNet 7124 0.7500 0.7375

dead fraction (block's second conv, post-BN where applicable):
block0 plain: 1.000 BN-only: 0.518 BN+res: 0.941
block1 plain: 0.875 BN-only: 0.526 BN+res: 0.740
block2 plain: 1.000 BN-only: 0.673 BN+res: 0.702
```

Not the tidy "combine both fixes for the best result" story Day 53's docstring half-predicted. PlainDeepConvNet still fails completely, unchanged from Day 53. PlainBNConvNet -- batch norm ALONE, no residual shortcuts -- reaches $\text{train_acc}=1.000/\text{test_acc}=0.975$, the best result any network in this two-day arc has reached. ResNetBNConvNet (the actual published ResNet basic block: BN and residual together) trains too, but far more unevenly and to a lower ceiling ($\text{test_acc}=0.738$). Adding a raw, unnormalized identity shortcut on top of a normalized branch, right before the final ReLU, changes the distribution the NEXT layer's batch norm has to renormalize -- at this lesson's scale, that combination trained less smoothly than batch norm alone, not more.



PlainBNConvNet (orange) climbs smoothly to ~0.98. ResNetBNConvNet (green) visibly oscillates and plateaus lower. PlainDeepConvNet (red) never leaves chance level.

This does not mean combining batch norm with residual connections is a bad idea in general -- real ResNet's success at ImageNet scale, over far more data and far deeper stacks (50-152+ layers), is well established. It means this lesson's own small-scale, honestly-run comparison doesn't reproduce the "more machinery is strictly better" story -- the same caveat Days 47, 49, 52, and 53 already taught with their own architectures.

5. What This Maps To in PyTorch

```
class ResNetBasicBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out)) # no relu yet
        return torch.relu(out + x) # add the shortcut, THEN relu
```

This IS the real ResNet (He et al., 2015) basic block, exactly as published. `nn.BatchNorm2d` also tracks a running mean/variance during training so it can normalize a single example at inference time without needing a batch of its own -- a detail this lesson's `spatial_batchnorm_forward` skips, following Day 40's own simplification (fine here since this series' `accuracy()` always evaluates a full array at once).

Interview Preparation -- Technical Questions

Q: How does spatial (conv) batch norm differ from dense-layer batch norm?

A: Dense batch norm treats each of D features independently, normalizing each across the batch (N values per feature). Spatial batch norm treats each CHANNEL as one feature, normalizing it using

statistics pooled over the batch AND both spatial axes ($N \times H \times W$ values per channel) -- because every spatial position in a given channel was produced by the SAME convolutional filter, so they share the same distribution.

Q: Why did the gradient check for dgamma/dbeta initially fail by a factor matching the batch size m?

A: Because dgamma/dbeta were computed as a raw sum over the pooled axes with no averaging, while every other parameter gradient in this codebase's layers (dW, db in conv2d_backward_mc and the FC layers) is explicitly divided by m to match cross_entropy_loss's own mean-over-batch convention. Without that same /m, gamma and beta received gradients m times too large relative to every other trainable parameter.

Q: Why is BN placed between the conv and the ReLU, rather than after the ReLU?

A: Batch norm is meant to control the DISTRIBUTION of a layer's pre-activation before a non-linearity clips or reshapes it. Placing BN after ReLU would normalize an already-half-zeroed distribution (ReLU output is never negative), which changes the statistics BN is trying to control and doesn't match how real architectures (including ResNet) place it.

Q: PlainBNConvNet outperformed ResNetBNConvNet in this lesson's trained comparison. Does this mean residual connections plus batch norm is a worse design than batch norm alone?

A: Not in general -- it means that at THIS lesson's scale (a 2-block synthetic dataset, 200 training images), adding an unnormalized identity shortcut on top of a normalized branch trained less smoothly than batch norm by itself. Real ResNet's combination of both was validated at ImageNet scale with far more data and far deeper networks. This is the same scale-dependence Days 47, 49, and 52 already found with their own architectures -- a real, verified structural idea doesn't automatically win on every metric at every scale.

Interview Preparation -- Theory Questions

Q: What problem did Ioffe & Szegedy (2015) originally propose batch normalization to solve?

A: Internal covariate shift -- the idea that each layer's input distribution keeps changing during training as earlier layers' weights update, forcing later layers to continuously re-adapt. By normalizing each layer's pre-activations, batch norm keeps that distribution more stable, which in practice allows higher learning rates and faster, more stable training.

Q: Why does nn.BatchNorm2d need to track a running mean/variance, when this lesson's implementation doesn't?

A: At inference time, a real deployed model often processes one example at a time, which has no batch statistics of its own to normalize with. A running (exponential moving) average of mean/variance, accumulated during training, lets the layer normalize consistently at inference without needing a batch. This lesson's accuracy() always evaluates a full array as one batch, so using that array's own current statistics (Day 40's original simplification) is not a practical problem here.

Q: This lesson found that PlainBNConvNet's gradient-flow ratio (1.53) actually exceeds 1.0, while BN+residual's ratio (1.15) is closer to 1.0 exactly. What does a ratio above 1.0 mean?

A: The stem-to-deepest-layer ratio compares the shallowest layer's gradient norm to the deepest layer's. A ratio above 1.0 means the shallowest layer actually received a LARGER gradient than the deepest one -- batch norm doesn't just prevent gradient from vanishing with depth, it can shift the

relative balance of how much gradient different layers receive, in ways beyond simple preservation.

Key Takeaway and What's Next

Key takeaway: Extending batch norm from dense layers to conv layers is a small mathematical change (pool statistics over batch+spatial axes, one gamma/beta per channel) with a large practical effect: it fully rescued training at the exact $\text{lr}=0.15$ that broke every network in Day 53, reaching $\text{test_acc}=0.975$. Combining it with residual shortcuts (the real ResNet basic block) also worked, but trained less smoothly and to a lower ceiling at this lesson's scale -- a genuine, measured result, not the "stack every fix together" story one might expect.

What's next: Day 55 closes out Week 8 with real image data pipeline efficiency -- why "dataloader" abstractions exist, and batching/shuffling patterns for handling data at a scale beyond what fits comfortably in memory.

Suggested watch order: 1) Batch Normalization ("batch norm") explained, 2) Batch Normalization | Internal Covariate Shift.